PROJECT: MULTI-AZ WORDPRESS DEPLOYMENT

Services: EC2, RDS (MySQL), EFS, ALB, CloudFront

Goal: Highly available WordPress deployment with content delivery optimization.

Skills: Shared storage (EFS), DNS, caching strategies.

Definition: This project demonstrates the deployment of a highly available and scalable WordPress website on AWS using a multi-AZ architecture. It integrates services such as Amazon EC2, Amazon RDS, Amazon EFS, Application Load Balancer (ALB) and CloudFront to improve reliability, performance, and content delivery. The project highlights how AWS services work together to build a resilient and production-ready web application.

Scope of the Project: This project focuses on building a highly available and scalable WordPress environment on AWS. It includes configuring EC2 instances, RDS databases, EFS shared storage, Route 53 DNS, and CloudFront content delivery. The implementation demonstrates cloud architecture best practices for load balancing, fault tolerance, and improved website performance.

Methodologies:

1. WordPress Deployment using EC2 + RDS + VPC – Method Used

- Uses AWS networking services including VPC, Subnets, Internet Gateway, Route Tables, and Security Groups.
- Deploys a WordPress application on an Amazon EC2 instance running Apache, PHP, and MySQL client.
- Uses Amazon RDS as a managed MySQL database instead of installing MySQL directly on the EC2 instance.
- Separates the application layer and database layer, improving security and scalability.
- Stores website data securely in Amazon RDS located within the VPC.
- Uses Security Groups to control communication between WordPress and the database.
- Provides high availability through deployment across multiple Availability Zones.
- Suitable for production-ready WordPress hosting environments.
- Reduces database administration effort through Amazon RDS automated backups and maintenance.
- Demonstrates cloud networking, compute, storage, and database integration within AWS.

2. WordPress Deployment using EC2 + Local MySQL – Alternative Method

- Installs both WordPress and MySQL database on the same EC2 instance.
- Requires manual installation and management of MySQL.
- Simpler to configure for testing and learning purposes.
- Lower initial setup complexity.
- Does not provide database isolation from the application server.
- Limited scalability because application and database resources share the same server.
- Requires manual database backup and maintenance.
- Less secure than using Amazon RDS.
- Suitable only for small-scale websites, testing environments, and proof-of-concept deployments.
- Not recommended for production workloads requiring scalability, reliability, and high availability.

Services Used in this Project:

1**. Amazon EC2 (Elastic Compute Cloud)**

- **Definition:** Amazon EC2 is a cloud computing service that provides virtual servers to run applications on AWS.
- **Purpose:** EC2 is used to host and run the WordPress website with Apache and PHP.
- **Role**: EC2 acts as the web server that handles user requests and delivers the WordPress website.

2. **Amazon RDS (Relational Database Service)**

- **Definition:** Amazon RDS is a managed database service that allows users to easily create, manage, and maintain relational databases in the cloud.
- **Purpose:** Amazon RDS is used to store and manage the WordPress MySQL database.
- **Role:** Amazon RDS acts as the backend database that stores website data such as posts, users, and settings.

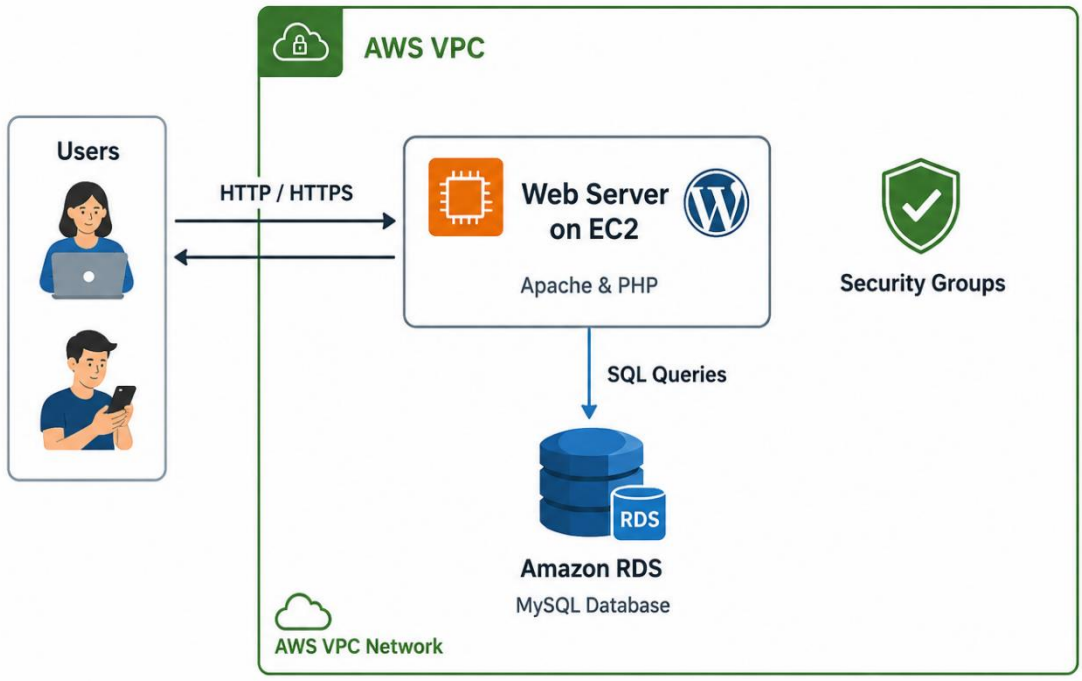
3. **Security Group**

- **Definition:** A Security Group is a virtual firewall in AWS that controls inbound and outbound network traffic for resources like EC2 and RDS.
- **Purpose:** The Security Group is used to allow necessary access such as HTTP, SSH, and database communication.
- **Role:** The Security Group ensures secure communication between the EC2 instance and the RDS database while protecting the server from unauthorized access.

4**. VPC (Virtual Private Cloud)**

- **Definition:** A VPC (Virtual Private Cloud) is a virtual network in AWS where cloud resources like EC2 and RDS are securely deployed.
- **Purpose:** The VPC provides the network environment for hosting the EC2 instance and RDS database.
- **Role:** The VPC enables secure communication between EC2 and RDS within the same network.

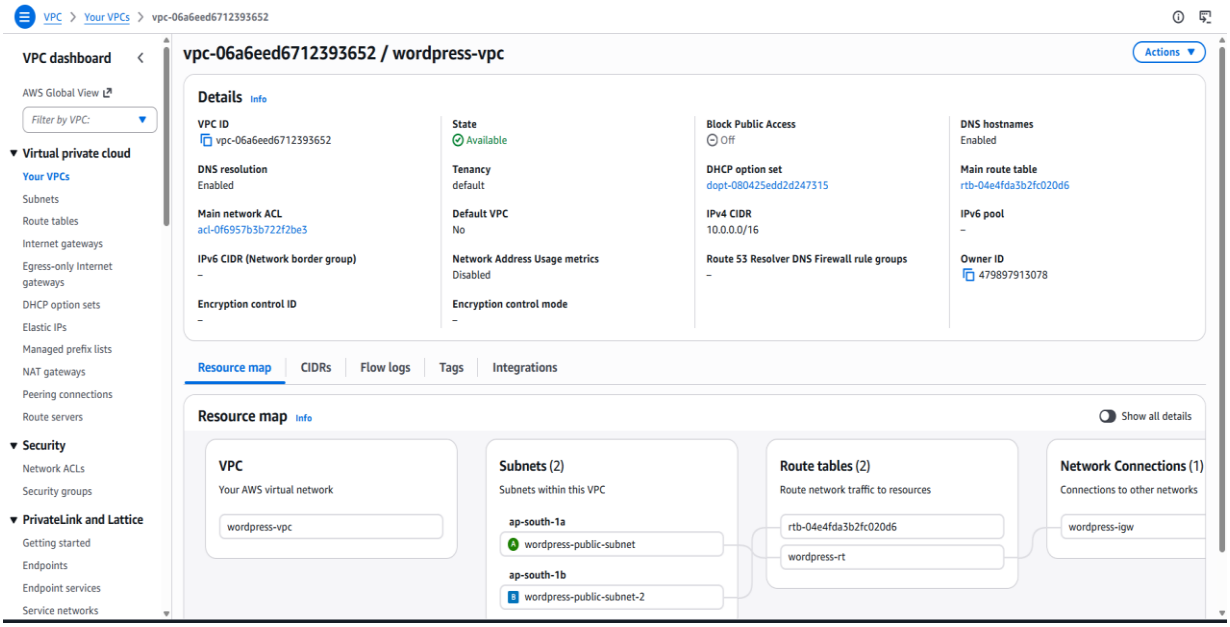
Architecture Diagram:



Implementation

STEP 1: Create VPC

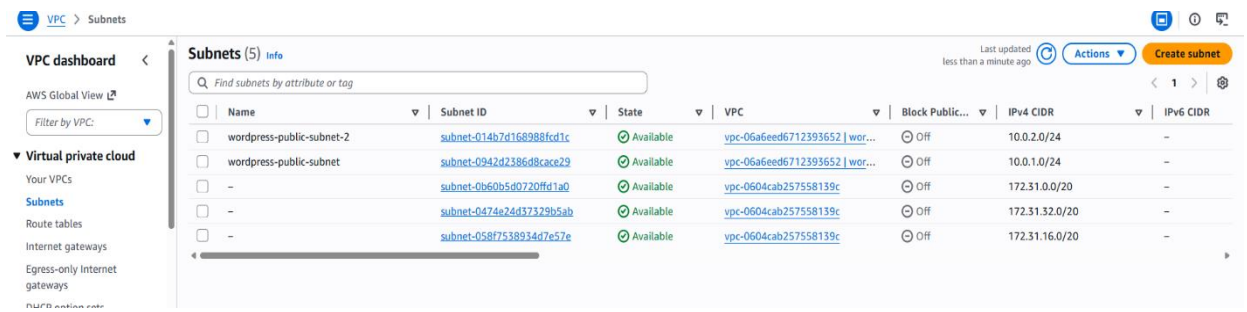
- Go to AWS Console → VPC → Create VPC.
- Resource Type: VPC Only.
- Name: wordpress-vpc.
- IPv4 CIDR Block: 10.0.0.0/16.
- Click Create VPC.
- Name: wordpress-vpc



STEP 2: Create Subnets

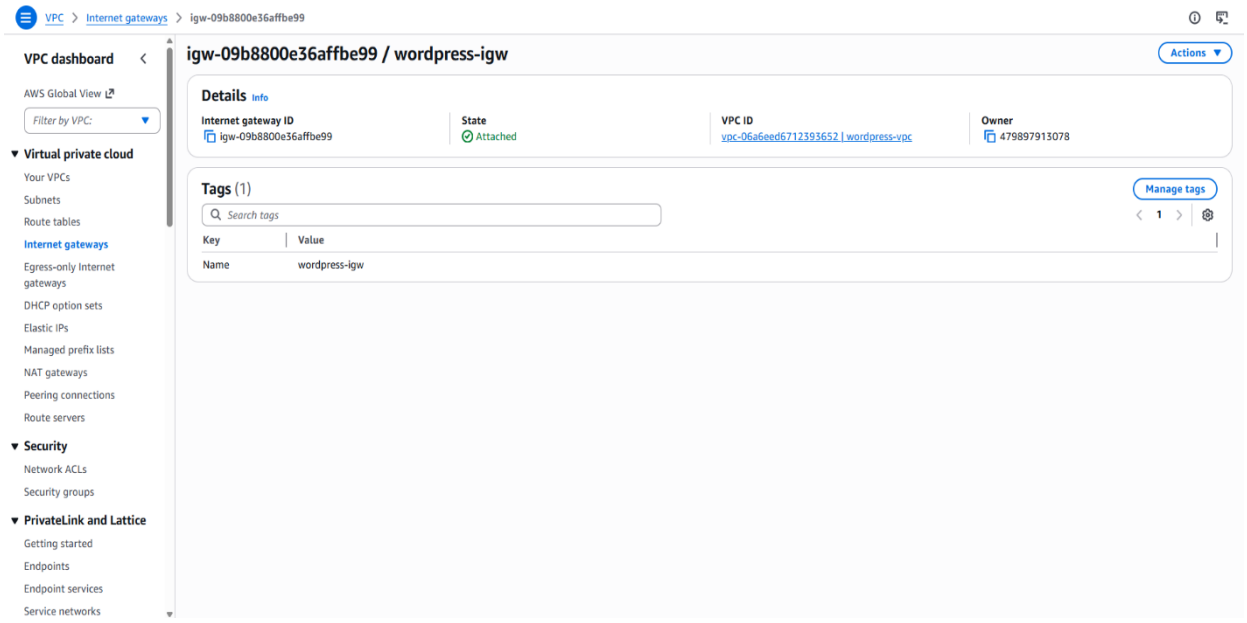
- Public Subnets

- Go to VPC → Subnets → Create Subnet.
- Select VPC: wordpress-vpc.
- Create Subnet 1:
- Name: wordpress-public-subnet-1
  - Availability Zone: ap-south-1a
  - CIDR Block: 10.0.1.0/24
- Create Subnet 2:
  - Name: wordpress-public-subnet-2
  - Availability Zone: ap-south-1b
  - CIDR Block: 10.0.2.0/24
- Click Create Subnets.



STEP 3: Create Internet Gateway

- Name: wordpress-igw
- Click Create Internet Gateway
- Select IGW: Actions - Attach to VPC
- Click on Attach



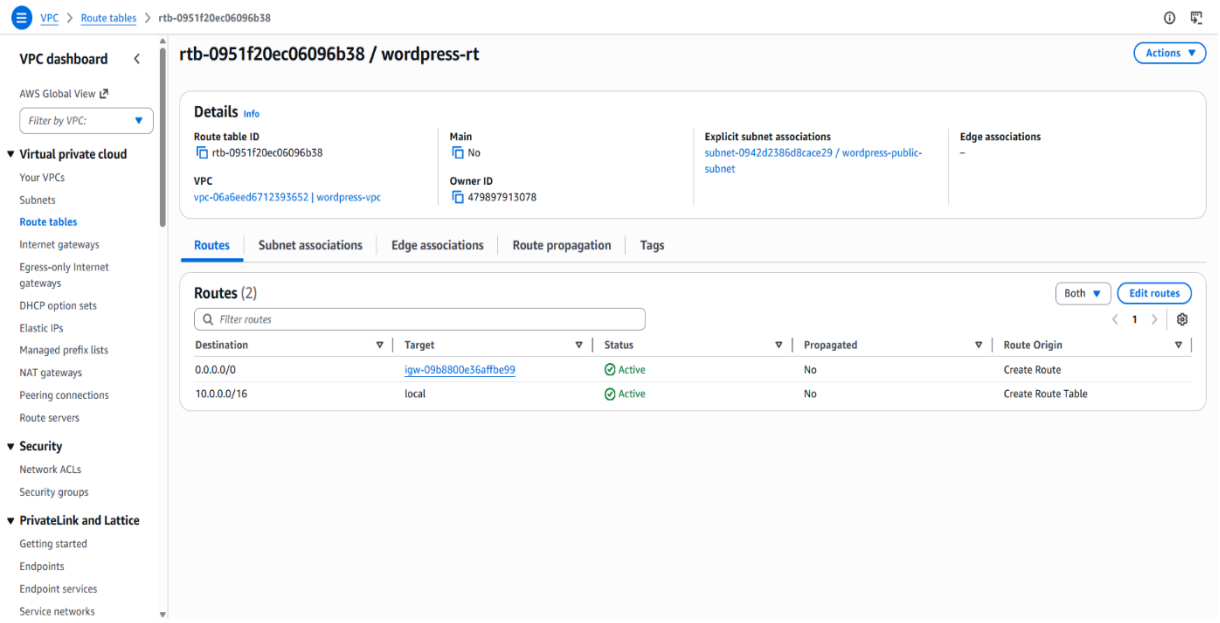
STEP 4: Create Public Route Table

- Name: wordpress-rt
- VPC: wordpress-vpc
- Click create route table

Add Route:

- Select wordpress-rt
- Routes: Edit routes
- Add routes:

➢ Destination: 0.0.0.0/0
➢ Target: Internet Gateway (wordpress-igw)
➢ Save routes

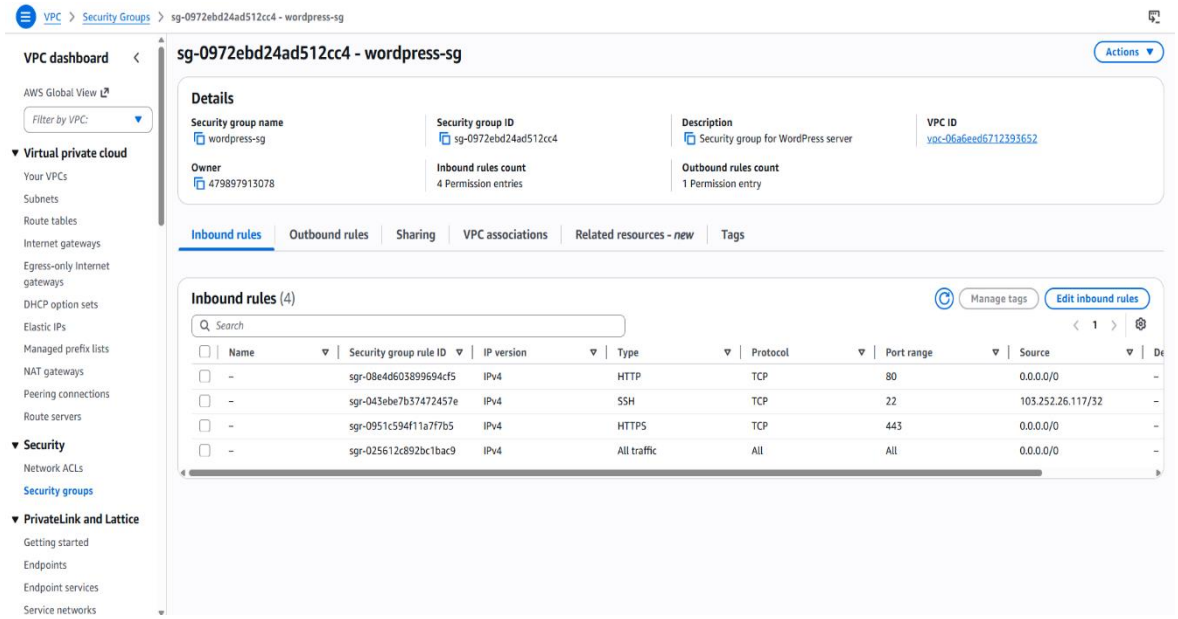


Step 5: Create Security Group

- Go to EC2 → Security Groups.
- Click Create Security Group.
- Name: wordpress-sg.
- Description: WordPress Security Group.
- VPC: wordpress-vpc.

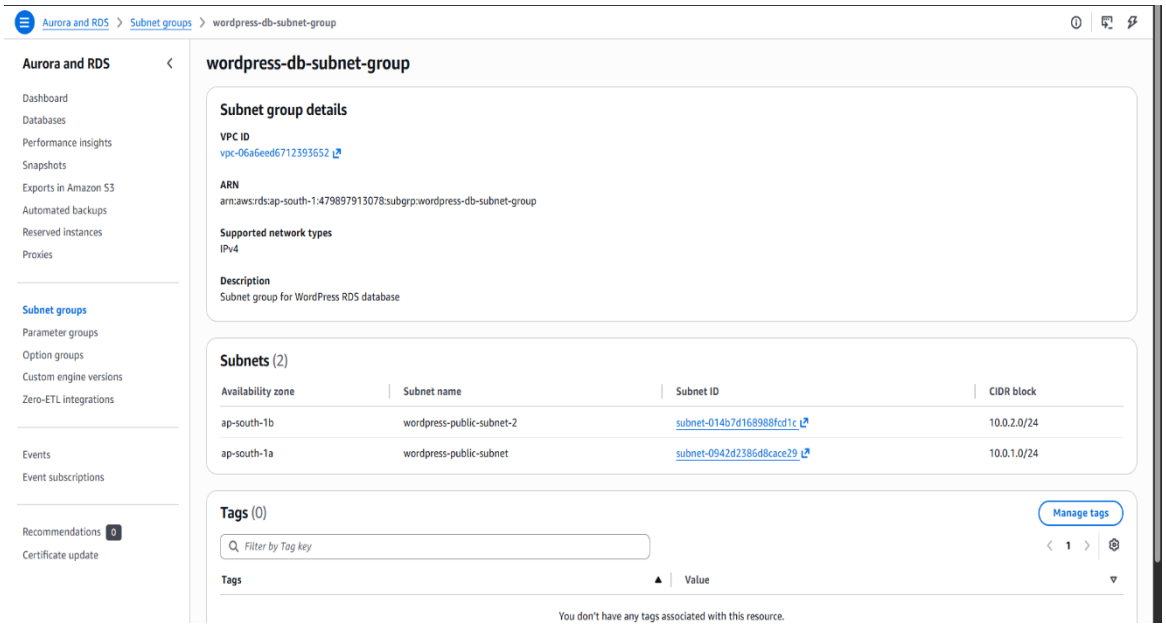
Inbound Rules

- SSH (22) → My IP
- HTTP (80) → Anywhere (0.0.0.0/0)
- HTTPS (443) → Anywhere (0.0.0.0/0)
- MySQL/Aurora (3306) → wordpress-sg
- Click Create Security Group.



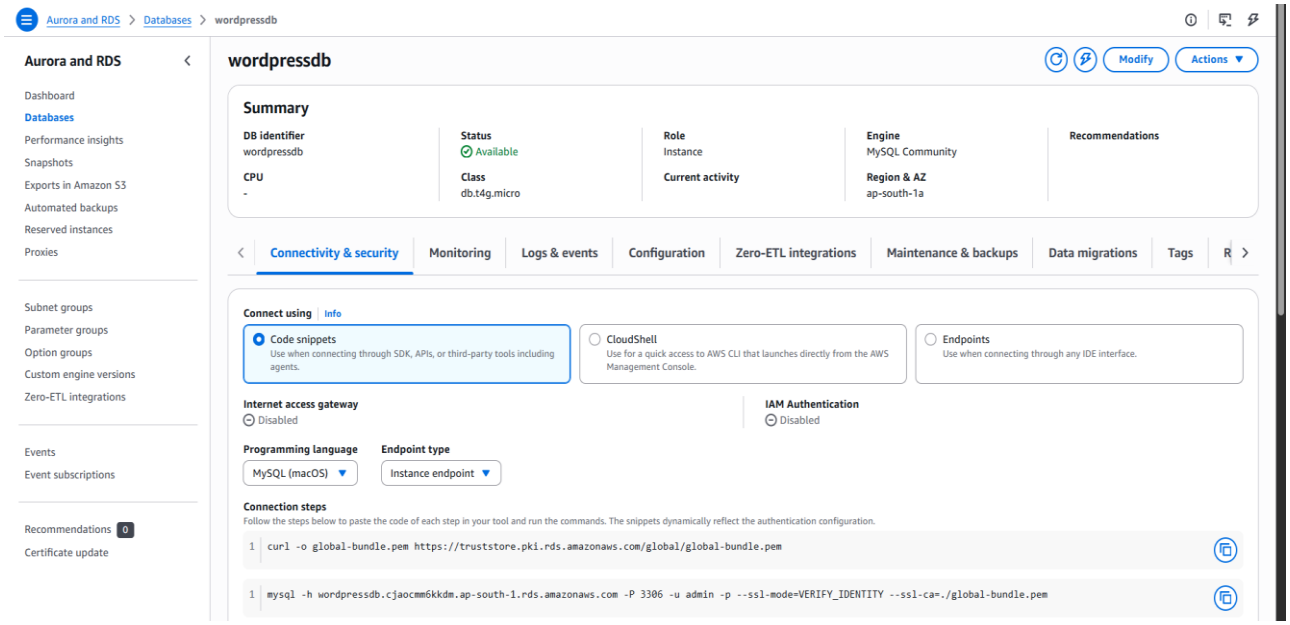
STEP 6: **Created Subnet Group**

- Go to RDS → Subnet Groups.
- Click Create DB Subnet Group.
- Name: wordpress-db-subnet-group.
- Description: WordPress Database Subnet Group.
- Select VPC: wordpress-vpc.
- Add both public subnets.
- Click Create.



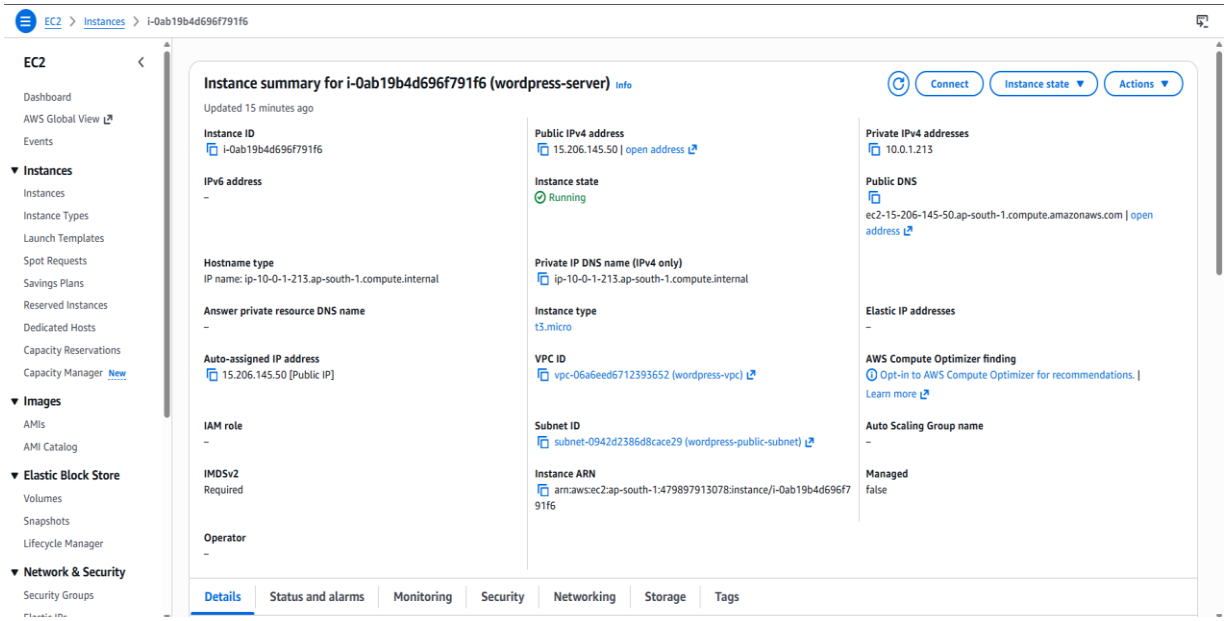
STEP 7: **Created Database**

- Go to RDS → Create Database.
- Creation Method: Standard Create.
- Engine: MySQL.
- Template: Free Tier.
- DB Instance Identifier: wordpress-db.
- Master Username: admin.
- Set Database Password.
- Select VPC: wordpress-vpc.
- Select DB Subnet Group: wordpress-subnet-group.
- Security Group: wordpress-sg.
- Disable Public Access.
- Click Create Database.
- Wait until database status becomes Available.



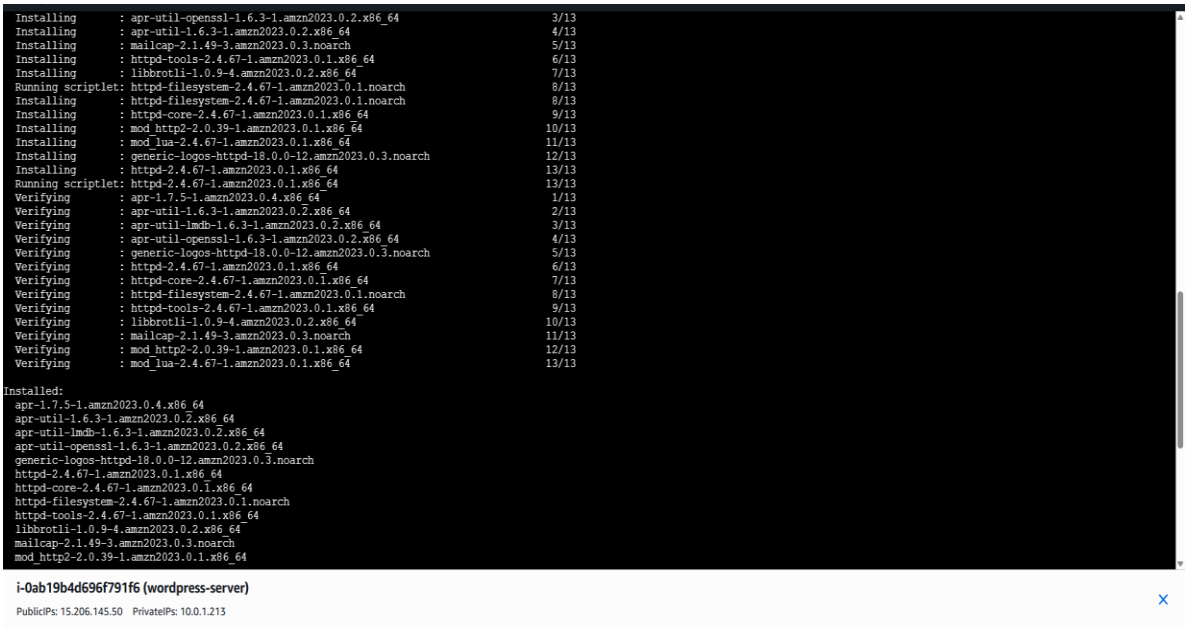
STEP 8: **Launch an instance**

- Go to EC2 → Launch Instance.
- Name: wordpress-server.
- AMI: Amazon Linux 2023.
- Instance Type: t2.micro or t3.micro.
- Key Pair: aws-keypair.
- Network: wordpress-vpc.
- Subnet: wordpress-public-subnet-1.
- Auto Assign Public IP: Enable.



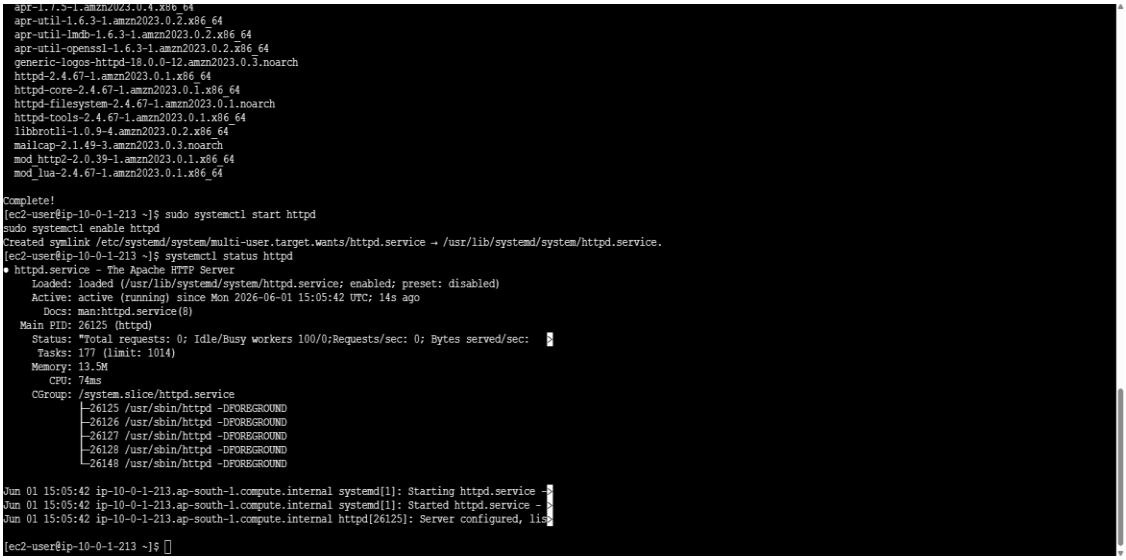
STEP 9: **Update the server and Install Apache + PHP + MySQL client**

- Connect to EC2 using SSH.
- Update the server:
- sudo yum update -y
- Install Apache:
- sudo yum install httpd -y
- Install PHP:
- sudo dnf install php php-mysqlnd php-fpm php-json php-cli php-common php-gd php-mbstring php-xml -y
- Install MySQL Client:
- sudo dnf install mariadb105 -y



STEP 10: Start Apache and Download wordpress

- Start Apache:
- sudo systemctl start httpd
- Enable Apache:
- sudo systemctl enable httpd
- Verify Status:
- sudo systemctl status httpd

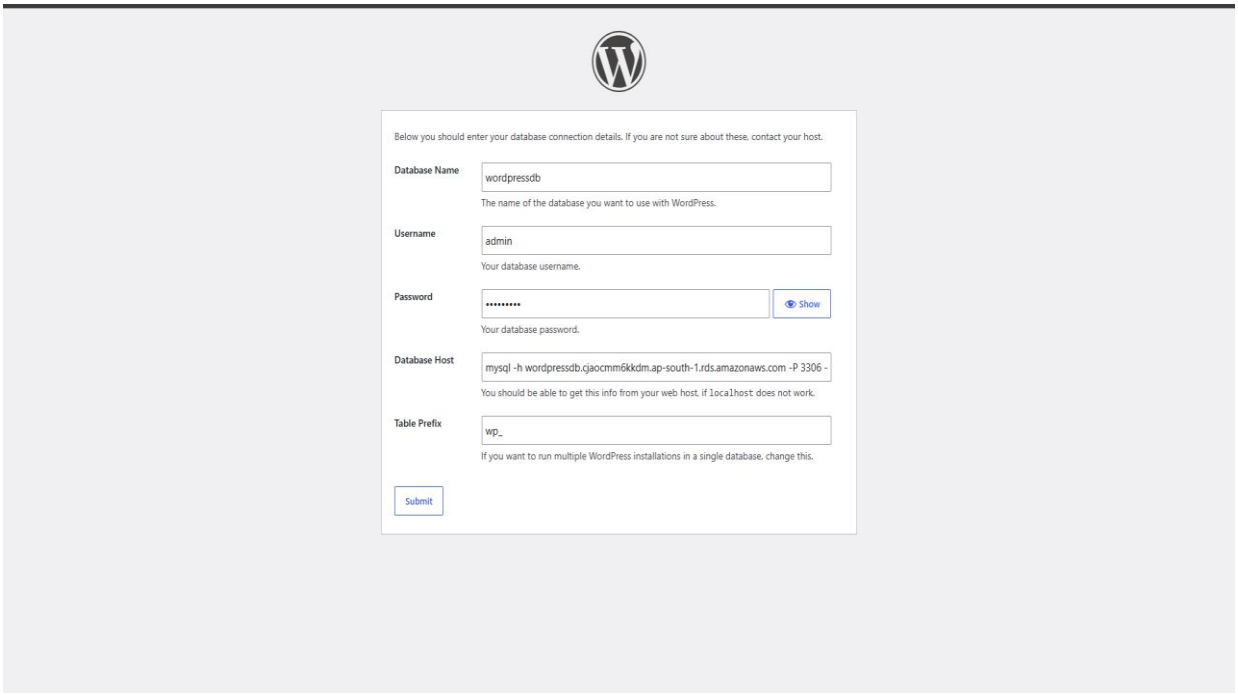


STEP 11 : **We can see wordpress installation page Navigate to Web Directory:**

cd /var/www/html

Download WordPress:

sudo wget https://wordpress.org/latest.tar.gz
Extract Files:
sudo tar -xvf latest.tar.gz
Copy Files:
sudo cp -r wordpress/* /var/www/html/
Set Permissions:
sudo chown -R apache /var/www/html
sudo chmod -R 755 /var/www/html
Restart Apache:
sudo systemctl restart httpd



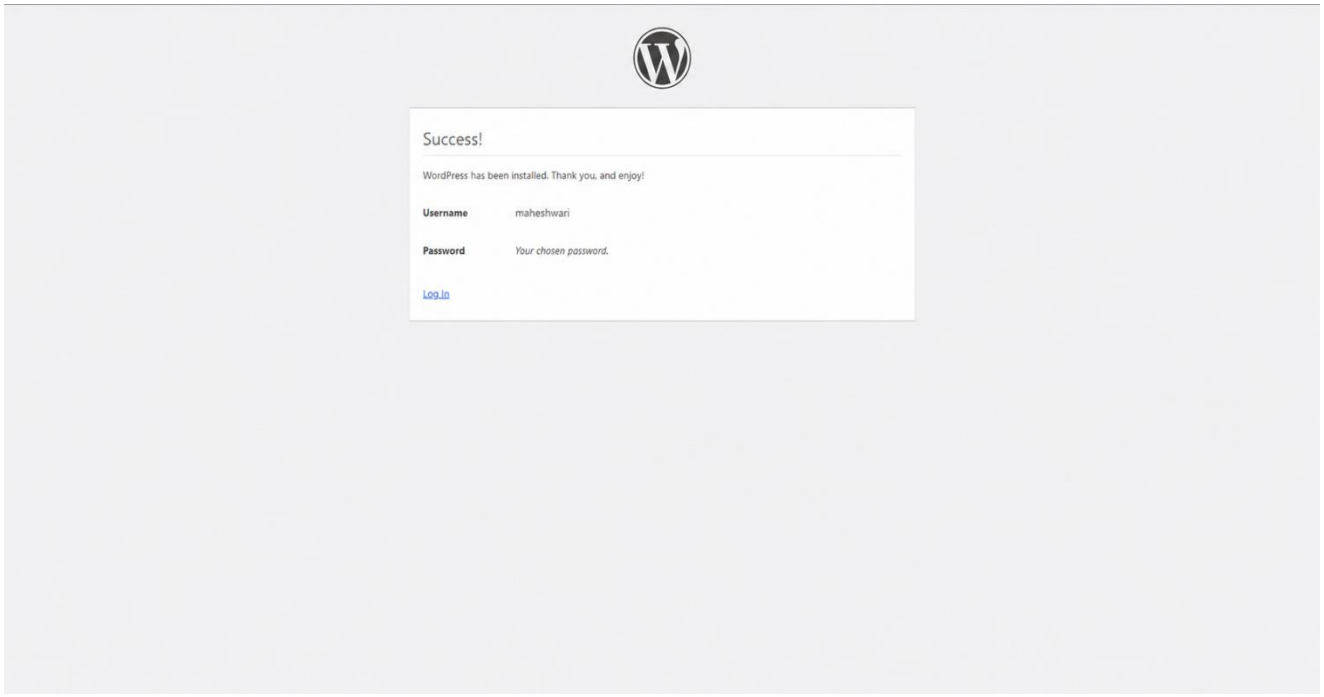
STEP 12: **We will connect wordpress → RDS database • Open Browser.**

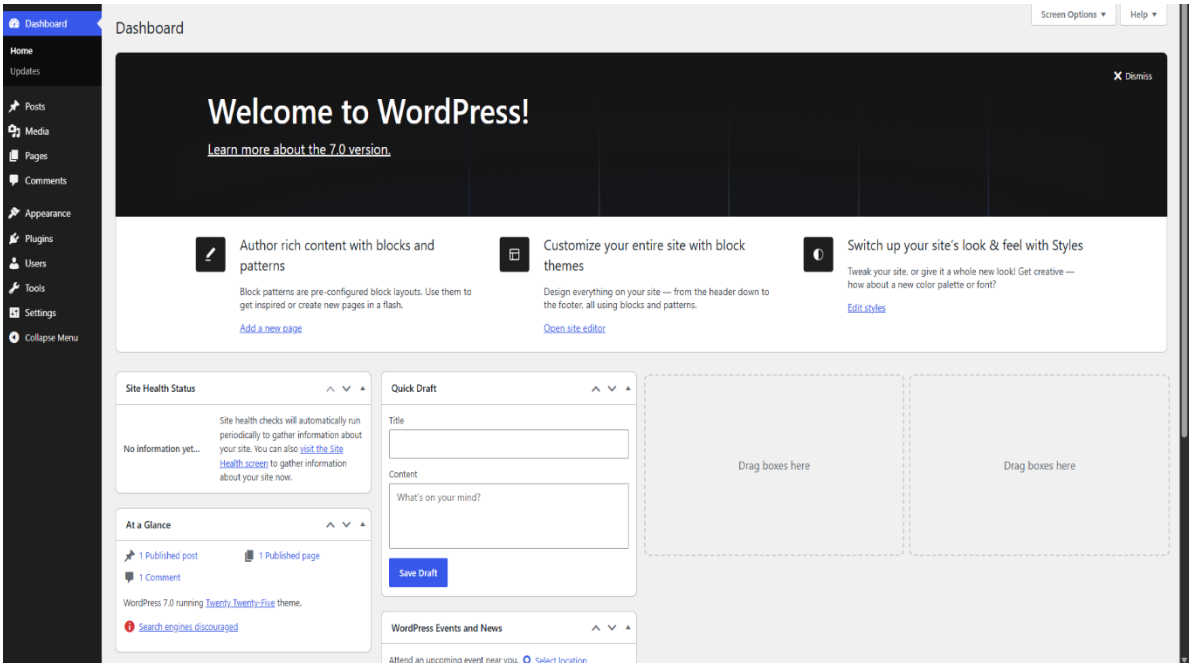
• Enter EC2 Public IP:
http://Public-IP
• Verify WordPress Setup Wizard appears.
• Select Language.
• Click Continue.

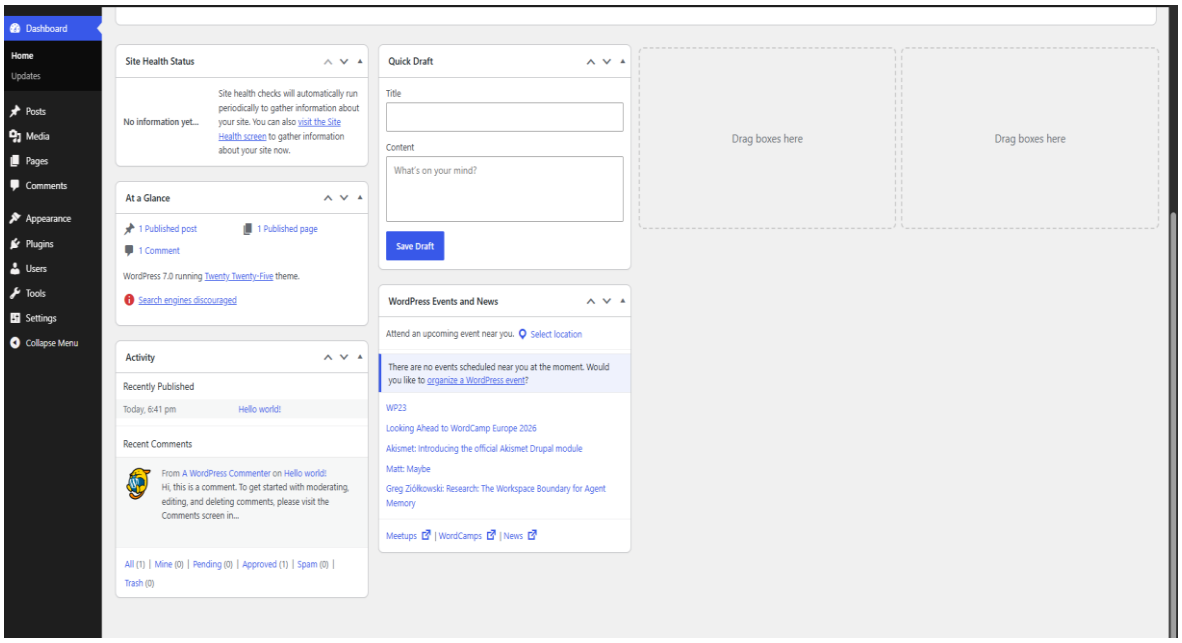
```
Server version: 8.0.46 Source distribution

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MySQL [(none)]> SHOW DATABASES;
+--------------------+
| Database           |
+--------------------+
| information_schema |
| mysql              |
| performance_schema |
| sys                |
| wordpressdb        |
+--------------------+
5 rows in set (0.018 sec)

MySQL [(none)]> DROP DATABASE wordpressdb;
Query OK, 12 rows affected (0.200 sec)

MySQL [(none)]> CREATE DATABASE wordpressdb;
Query OK, 1 row affected (0.007 sec)

MySQL [(none)]> SHOW DATABASES;
+--------------------+
| Database           |
+--------------------+
| information_schema |
| mysql              |
| performance_schema |
| sys                |
| wordpressdb        |
+--------------------+
5 rows in set (0.001 sec)

MySQL [(none)]> EXIT;
Bye
[ec2-user@ip-10-0-1-213 html]$ sudo systemctl restart httpd
[ec2-user@ip-10-0-1-213 html]$
```

i-0ab19b4d696f791f6 (wordpress-server)

PublicIPs: 15.206.145.50 PrivateIPs: 10.0.1.213







Troubleshooting:

1. WordPress Database Connection Error
• Issue: WordPress displayed “Error establishing database connection” during setup.
• Solution: Updated the wp-config.php file with the correct database name, username, password, and RDS endpoint.

2. EC2 Unable to Connect to RDS
• Issue: The EC2 instance could not connect to the MySQL database on Amazon RDS.
• Solution: Added a MySQL inbound rule (Port 3306) in the RDS security group and allowed access from the EC2 security group.

Conclusion:

This project successfully demonstrated the deployment of a highly available WordPress website on AWS using services such as Amazon EC2, Amazon RDS, EFS, ALB and CloudFront. The implementation provided hands-on experience in cloud infrastructure, database management, networking, and security. It also showed how AWS services can be integrated to create a scalable, reliable, and efficient web hosting environment. Overall, the project highlights the benefits of cloud-native architecture for deploying and managing modern web applications.